Gradient-based hyperparameter optimization

2026

Model selection: coherent inference

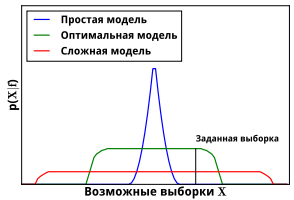
First level: select optimal parameters:

$$\mathbf{w} = \arg\max \frac{p(\mathfrak{D}|\mathbf{w})p(\mathbf{w}|\mathbf{h})}{p(\mathfrak{D}|\mathbf{h})},$$

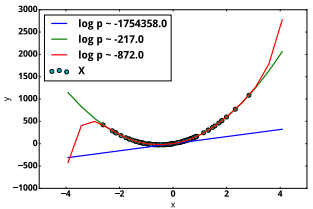
Second level: select optimal model (hyperparameters).

Evidence:

$$p(\mathfrak{D}|\mathbf{h}) = \int_{\mathbf{w}} p(\mathfrak{D}|\mathbf{w})p(\mathbf{w}|\mathbf{h})d\mathbf{w}.$$



Model selection scheme



Example: polynoms

Hyperparameters

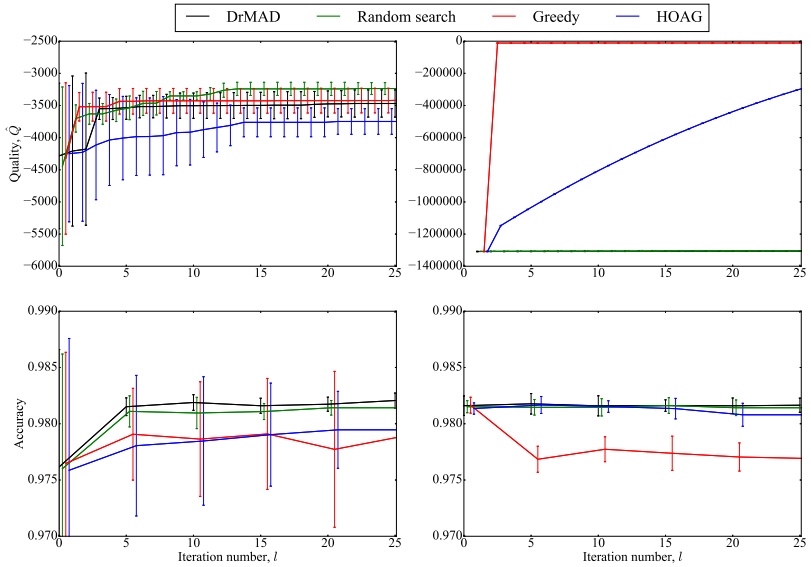
Definition

Prior for parameters $\mathbf{w}$ and structure $\boldsymbol{\Gamma}$ of the model $\mathbf{f}$ is a distrubution $p(\mathbf{W}, \boldsymbol{\Gamma}|\mathbf{h}) : \mathbb{W} \times \Gamma \times \mathbb{H} \to \mathbb{R}^+$, where $\mathbb{W}$ is a parameter space, Γ is a structure space.

Definition

Hyperparameters $\mathbf{h} \in \mathbb{H}$ of the models are the parameters of $p(\mathbf{w}, \boldsymbol{\Gamma}|\mathbf{h})$ (parameters of prior $\mathbf{f}$).

Experiment: MNIST



Experiment: MNIST

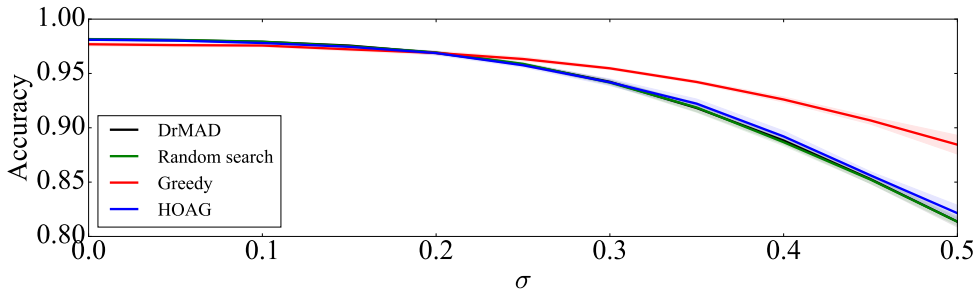
Adding Gaussian noise $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$:

Without noise

$\sigma = 0.1$

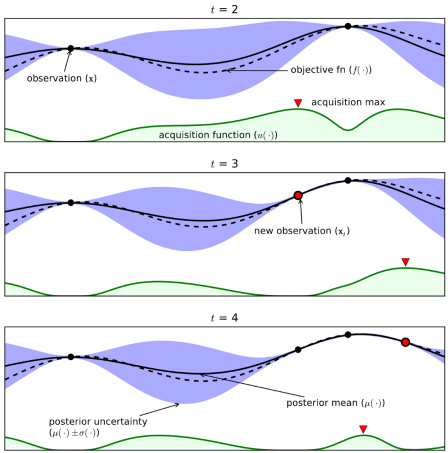
$\sigma = 0.25$

$\sigma = 0.5$



Non-gradient based methods: cons

- Need to run multiple times to get optimal hyperparameter values
- Computational complexity:
  - GP: $O(m^3)$, m is the number of observations.
  - TPE: $O(m \log m)$



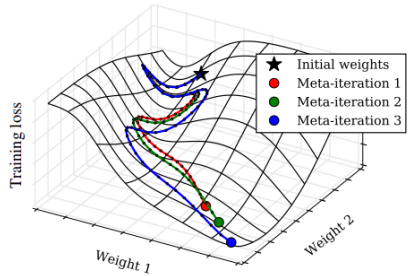
Shahriari et. al, 2016. GP example.

Gradient methods

Idea: Optimize hyperparameters using the full parameter optimization trajectory.
Pros:

- Hyperparameter optimization will consider the features of the parameter optimization.
- The complexity is linear on the number of hyperparameters.

Cons: the computational cost is very expensive. Also can be problems with numerical precision of reverse method.



Maclaurin et. al, 2015. Example.

Problem statement and notations

Given:

- $\mathbf{w} \in \mathbb{R}^N$: model parameters (or variational parameters $\boldsymbol{\theta}$);
- $\boldsymbol{\lambda} \in \mathbb{R}^H$: hyperparameters;
- differentiable train and validation losses $L_{\text{train}}(\mathbf{w}, \boldsymbol{\lambda}), L_{\text{val}}(\mathbf{w}, \boldsymbol{\lambda})$.

Hyperparameter optimization can be considered as a bi-level optimization problem:

$$\boldsymbol{\lambda}^* = \arg\min_{\boldsymbol{\lambda}^* \in \mathbb{R}^H} L_{\text{val}}(\mathbf{w}^*, \boldsymbol{\lambda}),$$

$$\text{s. t.} \quad \mathbf{w}^* = \arg\min_{\mathbf{W}^* \in \mathbb{R}^N} L_{\text{train}}(\mathbf{w}^*, \boldsymbol{\lambda}).$$

To use gradient-based optimization we need to find $\nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda})$.

Bi-level optimization: simple example

Classification problem with l_2-regularization:

$$\boldsymbol{\lambda}^* = \underset{\boldsymbol{\lambda}^* \in \mathbb{R}^H}{\arg\min}\, \text{CrossEntropy}\left(\mathbf{f}(\mathbf{X}_{\text{val}}, \mathbf{w}), \mathbf{y}_{\text{val}}\right)$$

$$\text{s. t.} \quad \mathbf{w}^* = \underset{\mathbf{w}^* \in \mathbb{R}^N}{\arg\min}\, \text{CrossEntropy}\left(\mathbf{f}(\mathbf{X}_{\text{train}}, \mathbf{w}), \mathbf{y}_{\text{train}}\right) + \lambda ||\mathbf{w}||^2.$$

Bi-level optimization: simple example

In general, $\boldsymbol{\lambda}$ can be presented in both problems:

$$\boldsymbol{\lambda}^* = \underset{\boldsymbol{\lambda}^* \in \mathbb{R}^H}{\arg\min}\, \text{CrossEntropy}\left(\mathbf{f}(\mathbf{X}_{\text{val}}, \mathbf{w}), \mathbf{y}_{\text{val}}\right) + \mathbf{G}(\boldsymbol{\lambda})$$

$$\text{s. t.} \quad \mathbf{w}^* = \underset{\mathbf{w}^* \in \mathbb{R}^N}{\arg\min}\, \text{CrossEntropy}\left(\mathbf{f}(\mathbf{X}_{\text{train}}, \mathbf{w}), \mathbf{y}_{\text{train}}\right) + \lambda ||\mathbf{w}||^2.$$

Problem statement: gradient-based optimization

Definition

An SGD operator T is an operator providing η steps of optimization:

$$\hat{\mathbf{w}} = T \circ T \circ \cdots \circ T(\mathbf{w}_0, \mathbf{h}) = T^{\eta}(\mathbf{w}_0, \mathbf{h}), \tag{1}$$

where

$$T(\mathbf{w}, \mathbf{h}) = \mathbf{w} - \lambda \nabla L_{\text{train}}(\mathbf{w}, \mathbf{h})|_{\hat{\mathfrak{D}}},$$

λ is a learning rate, $\mathbf{w}_0$ is an initial value of $\mathbf{w}$, $\hat{\mathfrak{D}}$ is a random subset of $\mathfrak{D}$.

Rewrite the optimization problem:

$$\mathbf{h}^* = \arg\max_{\mathbf{h} \in \mathbb{H}} L_{\text{val}}(T^{\eta}(\mathbf{w}_0, \mathbf{h})).$$

Gradient-Based Optimization of Hyperparameters, Bengio 2000

Simple case with quadratic train loss:

$$L_{\text{train}} = \mathbf{a}(\boldsymbol{\lambda}) + \mathbf{b}(\boldsymbol{\lambda})^{\mathsf{T}}\mathbf{w} + \frac{1}{2}\mathbf{w}^{\mathsf{T}}\mathbf{H}(\boldsymbol{\lambda})\mathbf{w}.$$

Solution for the inner problem:

$$\nabla_{\mathbf{w}} L_{\text{train}} = \mathbf{b} + \mathbf{H}(\boldsymbol{\lambda})\mathbf{w} = 0, \quad \mathbf{w}^*(\boldsymbol{\lambda}) = -\mathbf{H}^{-1}(\boldsymbol{\lambda})\mathbf{b}(\boldsymbol{\lambda}).$$

Assuming L_{val} depends on $\boldsymbol{\lambda}$ only via $\mathbf{w}$:

$$\underbrace{\nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda})}_{\text{gradient we need to compute}} = \underbrace{(\nabla_{\boldsymbol{\lambda}} \mathbf{w}^*(\boldsymbol{\lambda}))^{\mathsf{T}}}_{\text{best-response Jacobian}} \nabla_{\mathbf{w}} L_{\text{val}}, \quad \nabla_{\boldsymbol{\lambda}} w_i = -\sum_{j=1}^{H} \frac{\partial H^{-1}_{i,j}}{\partial \boldsymbol{\lambda}} b_j - \sum_{j=1}^{H} H^{-1}_{i,j} \frac{\partial b_j}{\partial \boldsymbol{\lambda}}.$$

The major problem is computing first term $\frac{\partial H^{-1}_{i,j}}{\partial \boldsymbol{\lambda}}$.

Gradient-Based Optimization of Hyperparameters, Bengio 2000

$$\underbrace{\nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda})}_{\text{gradient we need to compute}} = \underbrace{(\nabla_{\boldsymbol{\lambda}} \mathbf{w}^*(\boldsymbol{\lambda}))^{\mathsf{T}}}_{\text{best-response Jacobian}} \nabla_{\mathbf{w}} L_{\text{val}}, \quad \nabla_{\boldsymbol{\lambda}} w_i = -\sum_{j=1}^{H} \frac{\partial H^{-1}_{i,j}}{\partial \boldsymbol{\lambda}} b_j - \sum_{j=1}^{H} H^{-1}_{i,j} \frac{\partial b_j}{\partial \boldsymbol{\lambda}}.$$

2 ways to solve the problem:

- Naive approach using gradient over inverse Hessian matrix: $O(N^5)$.
- Solve the original equation $\nabla_{\mathbf{w}} L_{\text{train}} = \mathbf{b} + \mathbf{H}(\boldsymbol{\lambda})\mathbf{w}$ and backpropagate over the solution: $O(N^3)$.

General case, Bengio 2000

In general case we don't have an analytical solution for the inner problem and need to optimize it using numerical methods.

Implicit function theorem: the function $\mathbf{w}(\boldsymbol{\lambda})$ exits and differentiable near optimum $\mathbf{w}^*(\boldsymbol{\lambda})$:

$$\mathbf{F}(\mathbf{w}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda}) = \nabla_{\mathbf{w}} L_{\text{train}}|_{\boldsymbol{\lambda}, \mathbf{w}^*(\boldsymbol{\lambda})} = 0.$$

Differentiate $\mathbf{F}$ by $\boldsymbol{\lambda}$ at $\boldsymbol{\lambda}, \mathbf{w}^*(\boldsymbol{\lambda})$:

$$\frac{\partial \mathbf{F}}{\partial \mathbf{w}} \frac{\partial \mathbf{w}}{\partial \boldsymbol{\lambda}} + \frac{\partial \mathbf{F}}{\boldsymbol{\lambda}} = 0 \rightarrow$$

$$(\nabla_{\boldsymbol{\lambda}} \mathbf{w}(\boldsymbol{\lambda}))^{\mathsf{T}} \nabla^2_{\mathbf{w}} L_{\text{train}} + \nabla_{\mathbf{w}, \boldsymbol{\lambda}} L_{\text{train}} = 0.$$

We get a general closed form expression for the gradient:

$$\nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda}) = \nabla_{\boldsymbol{\lambda}} L_{\text{val}} + (\nabla_{\boldsymbol{\lambda}} \mathbf{w}(\boldsymbol{\lambda}))^{\mathsf{T}} \nabla_{\mathbf{w}} L_{\text{val}}$$

$$= \underbrace{\nabla_{\boldsymbol{\lambda}} L_{\text{val}}}_{\text{Direct gradient}} - \left(\nabla^2_{\mathbf{w}, \boldsymbol{\lambda}} L_{\text{train}}\right)^{\mathsf{T}} \underbrace{\left(\nabla^2_{\mathbf{w}} L_{\text{train}}\right)^{-1}}_{\text{Inverse Hessian}} \nabla_{\mathbf{w}} L_{\text{val}}.$$

Issues

$$\nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda}) = \underbrace{\nabla_{\boldsymbol{\lambda}} L_{\text{val}}}_{\text{Direct gradient}} - \left(\nabla^2_{\mathbf{w},\boldsymbol{\lambda}} L_{\text{train}}\right)^T \underbrace{\left(\nabla^2_{\mathbf{w}} L_{\text{train}}\right)^{-1}}_{\text{Inverse Hessian}} \nabla_{\mathbf{w}} L_{\text{val}}.$$

- $O(N^3)$ is better than $O(N^5)$, but still impractical for large-scale problems.
- We optimize hyperparameters $\boldsymbol{\lambda}$ for the converged values of $\mathbf{w}^*$.
  Optimization of some hyperparameters therefore is impossible:
  - parameter initial value;
  - learning rate.

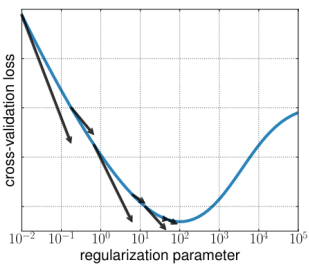
Hyperparameter optimization with approximate gradient, Pedregosa, 2016

Main idea: instead of computing $\left(\nabla^2_{\mathbf{w}} L_{\text{train}}\right)^{-1} \nabla_{\mathbf{w}} L_{\text{val}}$ we find a vector $\mathbf{v}$ that $\nabla^2_{\mathbf{w}} L_{\text{train}} \mathbf{v} \approx \nabla_{\mathbf{w}} L_{\text{val}}$.

In a loop:

- Solve the inner optimization problem up to tolerance ε: $|\mathbf{w}(\boldsymbol{\lambda}) - \mathbf{w}^*| < \varepsilon$.
- Solve the linear system $\nabla^2_{\mathbf{w}} L_{\text{train}} \mathbf{v} = \nabla_{\mathbf{w}} L_{\text{val}}$ w.r.t. $\mathbf{v}$ up to tolerance ε.
- Compute approximate gradient for $\boldsymbol{\lambda}$ as

$$\nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}(\boldsymbol{\lambda}), \boldsymbol{\lambda}) \approx \nabla_{\boldsymbol{\lambda}} L_{\text{val}} - (\nabla^2_{\mathbf{w},\boldsymbol{\lambda}} L_{\text{train}})^{\mathsf{T}} \mathbf{v}$$



HOAG, method analysis

Regularity conditions

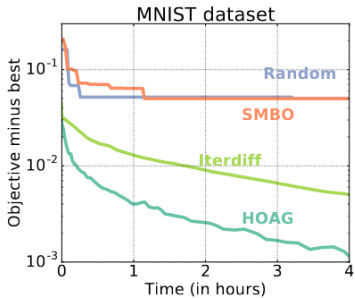
- First derivative of L_{train} and second derivates of L_{val} are Lipschitz smooth;
- Hessian $\nabla^2_{\mathbf{w}} L_{\text{train}}$ is invertible;
- Hyperparameter domain is convex.

HOAG properties

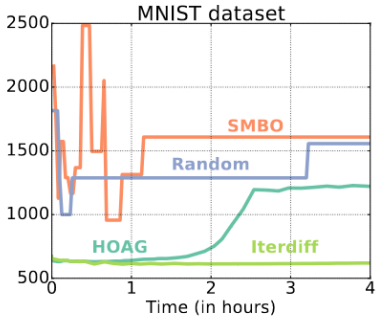
- For large number of iterations k the error in the gradient is bounded by a constant factor of ε_k: $||\nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}(\boldsymbol{\lambda}), \boldsymbol{\lambda} - \widetilde{\nabla}_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}(\boldsymbol{\lambda}), \boldsymbol{\lambda})|| = O(\varepsilon_k)$;
- If $\sum_{k=0}^{\infty} \varepsilon_k < \infty$, then $\lim_{k \to \infty} ||\widetilde{\nabla}_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}(\boldsymbol{\lambda}), \boldsymbol{\lambda})|| = 0$ when performing gradient descent.

HOAG, baseline comparison

Model: logistic regression trained on MNIST (7840 hyperparameters).



Validation dataset



Test dataset

We can see:

- The only methods worked well are gradient-based methods (HOAG and iterdiff);
- There is an overfitting on validation part of the dataset.

Approximation using Neumann series, Lorraine et al., 2020 [1]

Recap: Neumann series

Given an operator T, a Neumann series for it: $\sum_{i=0}^{\infty} T^i$.
If T is bounded, linear and its Neumann series converges, then

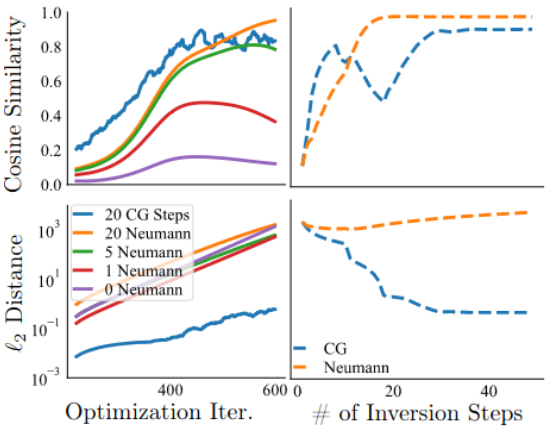
$$(I - T)^{-1} = \sum_{i=0}^{\infty} T^i.$$

We approximate inverse Hessian $\left(\nabla^2_{\mathbf{w}} L_{\text{train}}\right)^{-1}$ using K terms of the series:

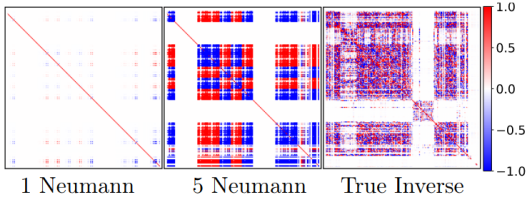
$$\left(\nabla^2_{\mathbf{w}} L_{\text{train}}\right)^{-1} \approx \sum_{i=0}^{K} I - \left(\nabla^2_{\mathbf{w}} L_{\text{train}}\right)^i,$$

[1] See Victor Pankratov's talk, 2021

Inverse Hessian approximation quality

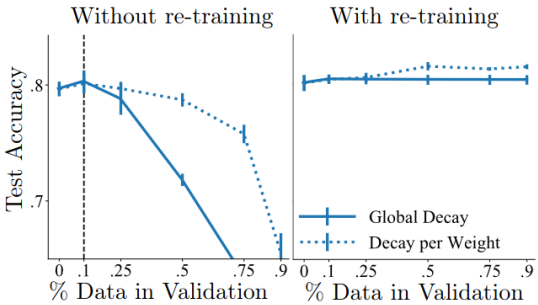


Inverse Hessian approximation using HOAG-like method and Neumann series



Inverse Hessian approximation for 1-layer NN, Boston dataset

Large-scale experiments



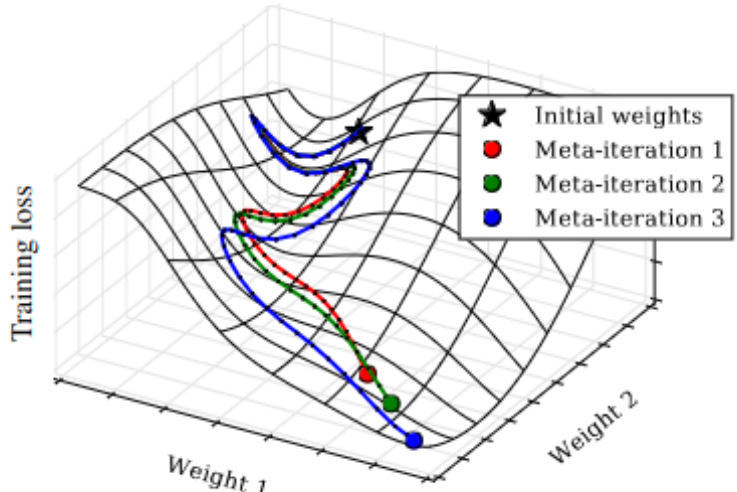
Logistic regression, MNIST

Method	Validation	Test	Time(s)
Grid Search	97.32	94.58	100k
Random Search	84.81	81.46	100k
Bayesian Opt.	72.13	69.29	100k
STN	70.30	67.68	25k
No HO	75.72	71.91	18.5k
Ours	69.22	66.40	18.5k
Ours, Many	**68.18**	**66.14**	18.5k

RNN with DropConnect coefficient optimization: up to 1.7M hyperparameters

RMAD, Maclaurin et al., 2015

Why not just propogate over the full optimization trajectory?



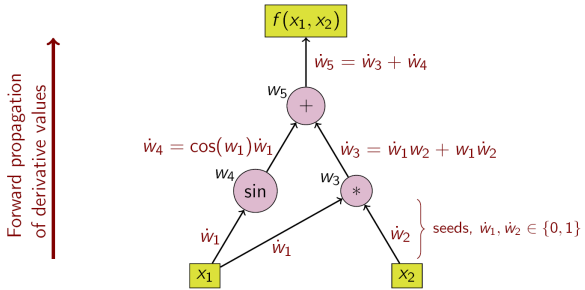
Forward-mode differentiation

Main idea:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial w_{n-1}} \frac{\partial w_{n-1}}{\partial x} = \frac{\partial y}{\partial w_{n-1}} \left(\frac{\partial w_{n-1}}{\partial w_{n-2}} \frac{\partial w_{n-2}}{x} \partial x \right) = \dots .$$

Example (wiki):

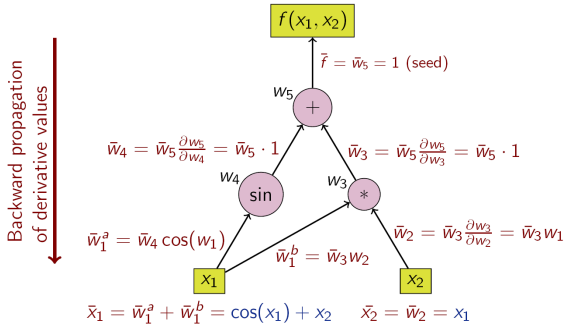
$$x_1 x_2 + \sin(x_1)$$



Reverse-mode differentiation

Idea:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial w_1}\frac{\partial w_1}{\partial x} = \left(\frac{\partial y}{\partial w_2}\frac{\partial w_2}{\partial w_1}\right)\frac{\partial w_1}{\partial x} = \dots.$$



SGD with momentum and reverse-mode gradient of SGD

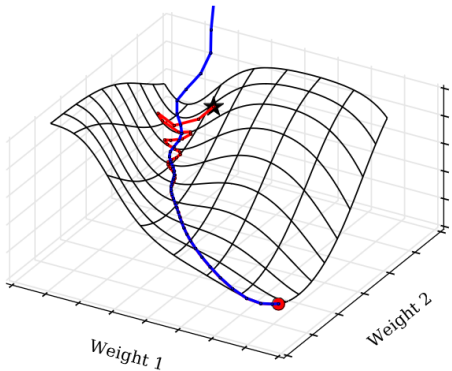
Stochastic Gradient Descent

input: initial $\mathbf{x}_1$, decays $\boldsymbol{\beta}$, learning rates $\boldsymbol{\alpha}$, loss function $L(\mathbf{x}, \boldsymbol{\theta}, t)$
initialize $\mathbf{v}_1 = \mathbf{0}$
for $t = 1$ **to** T **do**
  $\mathbf{g}_t = \nabla_{\mathbf{x}} L(\mathbf{x}_t, \boldsymbol{\theta}, t)$ ▷ evaluate gradient
  $\mathbf{v}_{t+1} = \beta_t \mathbf{v}_t - (1 - \beta_t)\mathbf{g}_t$ ▷ update velocity
  $\mathbf{x}_{t+1} = \mathbf{x}_t + \alpha_t \mathbf{v}_t$ ▷ update position
output trained parameters $\mathbf{x}_T$

Reverse-Mode Gradient of SGD

input: $\mathbf{x}_T$, $\mathbf{v}_T$, $\boldsymbol{\beta}$, $\boldsymbol{\alpha}$, train loss $L(\mathbf{x}, \boldsymbol{\theta}, t)$, loss $f(\mathbf{x})$
initialize $d\mathbf{v} = \mathbf{0}$, $d\boldsymbol{\theta} = \mathbf{0}$, $d\alpha_t = \mathbf{0}$, $d\beta = \mathbf{0}$
initialize $d\mathbf{x} = \nabla_{\mathbf{x}} f(\mathbf{x}_T)$
for $t = T$ **counting down to** 1 **do**
  $d\alpha_t = d\mathbf{x}^{\mathsf{T}} \mathbf{v}_t$
  $\mathbf{x}_{t-1} = \mathbf{x}_t - \alpha_t \mathbf{v}_t$ ▷ downdate position
  $\mathbf{g}_t = \nabla_{\mathbf{x}} L(\mathbf{x}_t, \boldsymbol{\theta}, t)$ ▷ evaluate gradient
  $\mathbf{v}_{t-1} = [\mathbf{v}_t + (1 - \beta_t)\mathbf{g}_t]/\beta_t$ ▷ downdate velocity
  $d\mathbf{v} = d\mathbf{v} + \alpha_t d\mathbf{x}$
  $d\beta_t = d\mathbf{v}^{\mathsf{T}}(\mathbf{v}_t + \mathbf{g}_t)$
  $d\mathbf{x} = d\mathbf{x} - (1 - \beta_t) d\mathbf{v} \nabla_{\mathbf{x}} \nabla_{\mathbf{x}} L(\mathbf{x}_t, \boldsymbol{\theta}, t)$
  $d\boldsymbol{\theta} = d\boldsymbol{\theta} - (1 - \beta_t) d\mathbf{v} \nabla_{\boldsymbol{\theta}} \nabla_{\mathbf{x}} L(\mathbf{x}_t, \boldsymbol{\theta}, t)$
  $d\mathbf{v} = \beta_t d\mathbf{v}$
output gradient of $f(\mathbf{x}_T)$ w.r.t $\mathbf{x}_1$, $\mathbf{v}_1$, $\boldsymbol{\beta}$, $\boldsymbol{\alpha}$ and $\boldsymbol{\theta}$

Naive approach: fails



Credits: https://www.robots.ox.ac.uk/seminars/Extra/2015_07_23_David_Duvenaud.pdf

Why reversible dynamics fails?

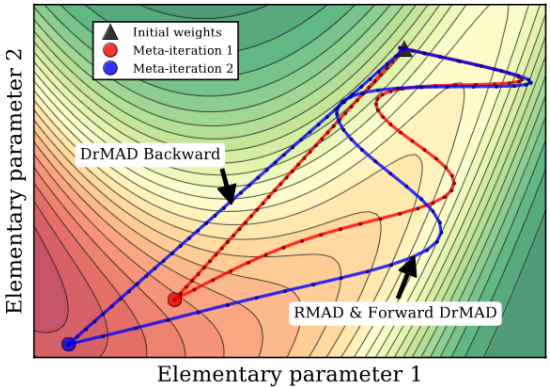
Update velocity step:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t - (1 - \beta)\mathbf{g}_t,$$

for the finite arithmetic this expression will destroy $\log_2 \beta$ bits of information.
Solution: make a scheme for storing destroyed bits of information to restore dynamics properly.
Conclusion: as a proof of concept, we really can restore the full trajectory, but this method requires momentum or similar techniques in optimization.

DRMAD, Fu et al., 2016

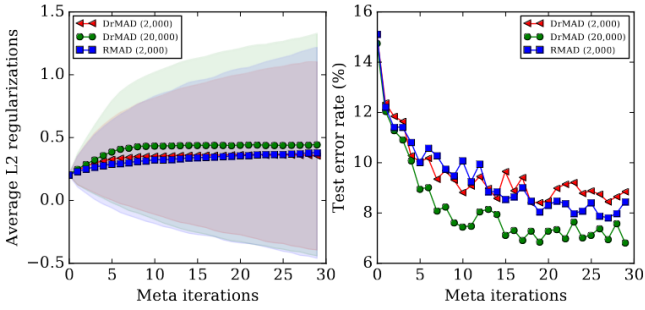
Instead of reversing dynamics replace it with linear trajectory of optimization.



DrMAD vs RMAD

Algorithm 2 Distilling reverse-mode automatic differentiation (DrMAD) of SGD.

inputs: initial parameter $\boldsymbol{w}_0$, learned parameter $\boldsymbol{w}_T$, training loss l_{train}, validation loss l_{valid}, weight decay γ, learning rate α
initialize $\mathrm{d}\boldsymbol{\lambda} \leftarrow 0$, $\mathrm{d}\boldsymbol{w} \leftarrow \nabla_{\boldsymbol{w}} l_{valid}$, $\beta_{T-1} \leftarrow 1 - \frac{1}{T}$
for $t = T - 1$ to 1 **do**
    $\boldsymbol{w}_{t-1} \leftarrow (1 - \beta_t)\boldsymbol{w}_0 + \beta_t \boldsymbol{w}_T$
    $\beta_{t-1} \leftarrow \beta_t - \frac{1}{T}$
    $\mathrm{d}\boldsymbol{v} \leftarrow \mathrm{d}\boldsymbol{v} + \alpha \mathrm{d}\boldsymbol{w}$
    $\mathrm{d}\boldsymbol{\lambda} \leftarrow \mathrm{d}\boldsymbol{\lambda} - (1 - \gamma)\mathrm{d}\boldsymbol{v}\nabla_{\boldsymbol{\lambda}}\nabla_{\boldsymbol{w}} l_{train}$
end for
output: gradient of l_{valid} w.r.t. $\boldsymbol{\lambda}$



T1-T2, Luketina et al., 2016

Main idea: instead of using full optimization trajectory, use just a one step.
The optimization can be considered as the following:

- Perform an optimization step for parameters $\mathbf{w}_{t+1}$
- Perform virtual next step optimization for parameters $\mathbf{w}_{t+1}(w_t, \boldsymbol{\lambda})$
- Calculate hyperparameter gradients from virtually compuated $\mathbf{w}_{t+1}$ as a function from $\mathbf{w}_t$:

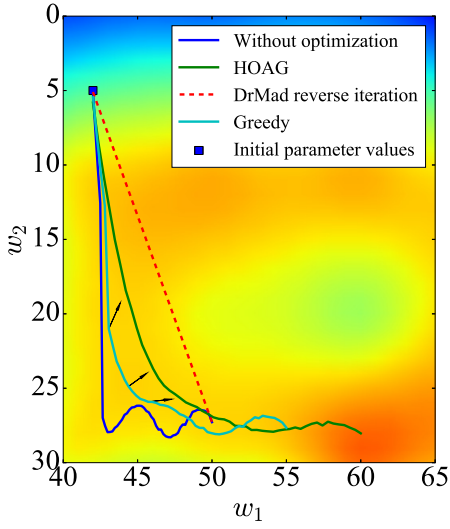
$$\nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda}) \approx \nabla_{\boldsymbol{\lambda}} L_{\text{val}}(\mathbf{w}_{t+1}(\mathbf{w}_t, \boldsymbol{\lambda}), \boldsymbol{\lambda}).$$

T1-T2, method properties

- The method works in a greedy way, there is no strict guarantees for convergence or optimality.
- The closed form of the hyperparameter gradient:

$$\nabla_{\mathbf{w}} L_{\text{val}} \nabla_{\lambda,\mathbf{w}} L_{\text{train}}.$$

  This expression coincides with general closed form when $\left(\nabla^2_{\mathbf{w}} L_{\text{train}}\right)^{-1} = \mathbf{I}$.
- The method complexity is $O(|\mathbf{w}| \cdot |\boldsymbol{\lambda}|)$, but can be improved by using finite difference approximation $O(|\mathbf{w}| + |\boldsymbol{\lambda}|)$, (Liu 2019 et al).
- We can go further and optimize hyperparameter even without making a virtual step, (Liu 2019 et al).


Without optimization
HOAG
DrMad reverse iteration
Greedy
Initial parameter values
0
5
10
15
20
25
30
40
45
50
55
60
65
w_1
w_2

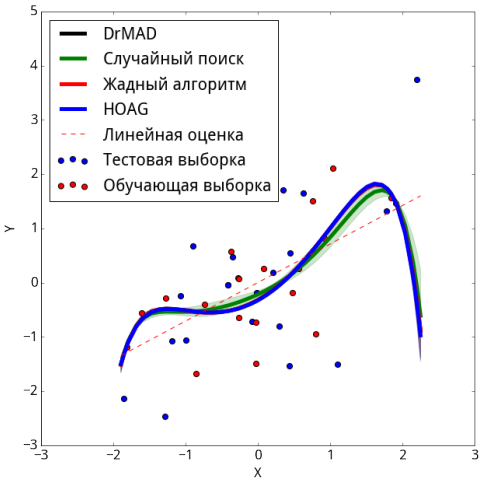
Algorithm comparison

Algorithm	+	-
Random search	Easy to implement	Curse of dimmensionality
Greedy optimization	Easy to implement, can be run inside common gradient optimization loop	Non-optimality.
HOAG	Fast convergence.	Results quality depends on the the linear system solution precision $\mathbf{H}(\boldsymbol{\theta})\boldsymbol{\lambda} = \nabla_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \mathbf{h})$.
DrMAD	Considers optimization features. Can be used for metaparameter optimization.	Not very tobust. Strong assumptions about linearity of the optimization trajectory.

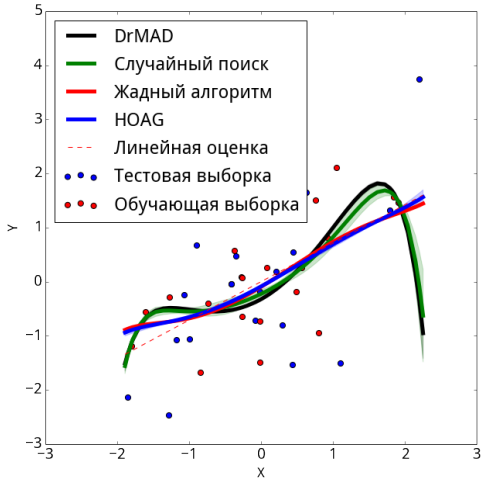
Overall comparison

Method	Steps	Eval.	Hypergradient Approximation
Exact IFT	∞	$\mathbf{w}^*(\boldsymbol{\lambda})$	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} - \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \times \left[\frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}}\right]^{-1} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \boldsymbol{\lambda}}\Big\vert_{\mathbf{w}^*(\boldsymbol{\lambda})}$
Unrolled Diff. [Maclaurin et al., 2015]	i	$\mathbf{w}_0$	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} - \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \times \sum_{j \leq i} \left[\prod_{k<j} I - \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}}\Big\vert_{\mathbf{w}_{i-k}}\right] \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \boldsymbol{\lambda}}\Big\vert_{\mathbf{w}_{i-j}}$
L-Step Truncated Unrolled Diff.	i	$\mathbf{w}_L$	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} - \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \times \sum_{L \leq j \leq i} \left[\prod_{k<j} I - \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}}\Big\vert_{\mathbf{w}_{i-k}}\right] \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \boldsymbol{\lambda}}\Big\vert_{\mathbf{w}_{i-j}}$
Larsen et al. [1996]	∞	$\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})$	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} - \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \times \left[\frac{\partial \mathcal{L}_T}{\partial \mathbf{w}} \frac{\partial \mathcal{L}_T}{\partial \mathbf{w}}^T\right]^{-1} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \boldsymbol{\lambda}}\Big\vert_{\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})}$
Bengio [2000]	∞	$\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})$	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} - \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \times \left[\frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}}\right]^{-1} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \boldsymbol{\lambda}}\Big\vert_{\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})}$
$T1-T2$ [27]	1	$\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})$	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} - \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \times [I]^{-1} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \boldsymbol{\lambda}}\Big\vert_{\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})}$
Ours	i	$\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})$	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} - \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \times \left(\sum_{j \leq i} \left[I - \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}}\right]^j\right) \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \boldsymbol{\lambda}}\Big\vert_{\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})}$
Conjugate Gradient (CG) $\approx$	-	$\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})$	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} - \left(\arg\min_{\mathbf{x}} \Vert \mathbf{x} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}} - \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \Vert\right) \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \boldsymbol{\lambda}}\Big\vert_{\widehat{\mathbf{w}^*}(\boldsymbol{\lambda})}$
Hypernetwork	-	-	$\frac{\partial \mathcal{L}_V}{\partial \boldsymbol{\lambda}} + \frac{\partial \mathcal{L}_V}{\partial \mathbf{w}} \times \frac{\partial \mathbf{w}_\phi^*}{\partial \boldsymbol{\lambda}}$ where $\mathbf{w}_\phi^*(\boldsymbol{\lambda}) = \arg\min_\phi \mathcal{L}_T(\boldsymbol{\lambda}, \mathbf{w}_\phi(\boldsymbol{\lambda}))$
Bayesian Optimization	-	-	$\frac{\partial \mathbb{E}[\mathcal{L}_V^*]}{\partial \boldsymbol{\lambda}}$ where $\mathcal{L}_V^* \sim$ Gaussian-Process$(\{\boldsymbol{\lambda}_i, \mathcal{L}_V(\boldsymbol{\lambda}_i, \mathbf{w}^*(\boldsymbol{\lambda}_i))\})$

Experiment: polynoms



CV



Evidence

References

- Bengio, Yoshua. "Gradient-based optimization of hyperparameters."Neural computation 12.8 (2000): 1889-1900.
- Bishop C. M. Pattern recognition //Machine learning. – 2006. – Т. 128. – №. 9.
- Bakhteev O. Y., Strijov V. V. Comprehensive analysis of gradient-based hyperparameter optimization algorithms //Annals of Operations Research. – 2020. – Т. 289. – №. 1. – С. 51-65.
- Graves A. Practical variational inference for neural networks //Advances in neural information processing systems. – 2011. – Т. 24.
- Bergstra et al., Random Search for Hyper-Parameter Optimization, 2012
- Dougal Maclaurin et. al, Gradient-based Hyperparameter Optimization through Reversible Learning, 2015
- Jelena Luketina et. al, Scalable Gradient-Based Tuning of Continuous Regularization Hyperparameters, 2016
- Jie Fu et. al, DrMAD: Distilling Reverse-Mode Automatic Differentiation for Optimizing Hyperparameters of Deep Neural Networks, 2016
- Fabian Pedregosa, Hyperparameter optimization with approximate gradient, 2016
- Bobak Shahriari et. al, Taking the Human Out of the Loop: A Review of Bayesian Optimization, 2016
- Liu H., Simonyan K., Yang Y. Darts: Differentiable architecture search //arXiv preprint arXiv:1806.09055. – 2018.
- Lorraine, J., Vicol, P., & Duvenaud, D. (2020, June). Optimizing millions of hyperparameters by implicit differentiation. In International Conference on Artificial Intelligence and Statistics (pp. 1540-1552). PMLR. (see also Victor Pankratov's slides, BMM-2021)
- Franceschi, Luca, et al. "Forward and reverse gradient-based hyperparameter optimization."International Conference on Machine Learning. PMLR, 2017.

Organizational issues

- Talks: 1 talk and blogpost are still required
- You can also make an additional talk for 1 extra-score
  - Talk priority: first tak has higher priority
  - But only one first talk per classes

Next hometask: presentation

- For all teams and their members: make presentations of your projects
- The presentation must cover
  - Project description (maybe more detailed than I gave to you)
  - Name of the project library
  - Scheme of the project (what will be the classes, how it will be integrated, what's the stack)
  - Brief algorithm description (from 1 to 4 slides for all the algorithms, other people must be able to understand the idea of all the algorithms)
  - **Idea** for proof of concept
- Time limit: 10 min

Next hometask: for people who are wrapping the library

- Create a repository in intsystems
- Keep in mind the future library must support documentation deploy and auto-testing. You can make the repository by yourself OR
  - Use Andrey Grabovoy's template: https://github.com/intsystems/ProjectTemplate
    - ⋆ Please turn off autodeploy of github pages
  - Use my template: https://github.com/intsystems/SoftwareTemplate-simplified
  - Use cookiecutter https://cookiecutter-data-science.drivendata.org/
  - Use any other template, see for example: https://github.com/LauzHack/pytorch_project_template
- Think about stack:
  - Codestyle (linters?)
  - Documentation engines (mkdocs? shpinx?)
  - Test libraries (built-in unittest? pytest?)
- Please make it w.r.t. to the manual

RTFM

Please make it w.r.t. to the manual

Next hometask: for people who are planning the library

- Create a document in the repository with the following information (the same as in the presentation, but maybe with more details)
  - Project name
  - Architecture of the project: what classes must be implemented? How they should interact?
  - Describe all the public functions and class methods you are planing to implement, with annotations.
  - What are the libraries you are planning to use and/or integrate?
- In perfect case, the member who is implementing the algorithm can write the code just by your architecture description.
- Note, the document can be improved/changed in the future, but I will score you and other members of the team on the correspondence of the proposed structure and the final algorithm implementation.